Изменение вектора развития

Oct 12, 2026 · @aron

1. Что изменилось и почему

Предметом ВКР становится навигация, а не инвентаризация. Примерная формулировка темы: «Выполнение навигационного задания на складе с использованием статичных меток (ArUco-маркеры и др.)». Точное название преподаватель сообщит позже.

Замечания преподавателя:

- **Инвентаризация — это задача, а не тема.** Считывание коробок — прикладной сценарий. Предмет исследования — как дрон узнаёт, где он, и как попадает в заданную точку склада.
- **SLAM не должен быть основой.** Склад — статичная среда, его план известен заранее. Строить карту на лету незачем.
- **Визуальная одометрия в проходах работает плохо.** При 1–1,5 м/с и стеллажах в 1–1,5 м от камеры картинка смещается слишком быстро. VO хороша там, где объекты далеко: обследование леса, открытые пространства.
- **Основа локализации — ArUco-маркеры.** Маркер кодирует биты чёрными и белыми модулями. При фиксированном размере маркера: крупнее модуль — меньше бит, но дальше читается.

Порядок работы: сначала навигация по складу (между стеллажами), затем навигация у стеллажа (подлёт к коробкам). Инвентаризация остаётся как демонстрация того, что навигационное задание выполнено.

2. Почему SLAM и визуальная одометрия не должны быть основой

Преподаватель прав по трём причинам: карта склада известна, близкие стеллажи дают быстрый оптический поток и смаз, а одинаковые проходы путают распознавание мест. Поэтому основа — локализация по известным меткам, а одометрия нужна лишь как «мост» на 1–3 с между ними.

2.1. SLAM решает лишнюю задачу

SLAM описывается двумя уравнениями (Gao & Zhang, ф. 1.1–1.2): движения и наблюдения.

$$x_k = f(x_{k-1}, u_k, w_k), \qquad z_{k,j} = h(y_j, x_k, v_{k,j})$$

Здесь x_k — поза дрона, u_k — показания IMU/одометрии, y_j — положения ориентиров, $z_{k,j}$ — наблюдения. SLAM ищет и позы, и карту:

$$\text{SLAM:}\; p(x_{1:k},\, y_{1:N} \mid z_{1:k}, u_{1:k}) \qquad \text{Локализация:}\; p(x_k \mid z_{1:k}, u_{1:k}, y_{1:N})$$

На складе y_j известны: координаты меток измеряются при монтаже и берутся из плана стеллажей. Неизвестных остаётся шесть (поза), а не 6 + 3N. Уходят построение карты, замыкание петель и рост ковариации от ориентиров в векторе состояния (книга, разд. 8.1.4, п. 3).

2.2. Близкие стеллажи — быстрый поток и смаз

Из пинхол-модели (ф. 4.5) $u = f_x \cdot X/Z + c_x$. Пусть камера движется вдоль фасада со скоростью v, расстояние до фасада Z = d. Дифференцируем по времени:

$$\dot u = \frac{f_x\, v}{d}, \qquad \Delta u_{\text{кадр}} = \frac{f_x\, v}{d\, F}, \qquad b_{\text{смаз}} = \frac{f_x\, v\, \tau}{d}$$

F — частота кадров, τ — выдержка. Главное: сдвиг и смаз обратно пропорциональны расстоянию d. Расчёт для широкоугольной VIO-камеры ($f_x \approx 320$ px, как у 1280×800 с HFOV ≈ 127°), v = 1,5 м/с, F = 30 Гц, τ = 8 мс (типично для складского света):

Сцена	d, м	Сдвиг за кадр, px	Смаз, px
Узкий проход, стеллаж рядом	1	16	3,8
Широкий проход	3	5,3	1,3
Обследование леса, открытое пространство	20	0,8	0,2

Почему это важно. Трекер Лукаса–Канаде (ф. 7.1–7.9) предполагает постоянство яркости и малый сдвиг:

$$I(x+dx,\, y+dy,\, t+dt) = I(x, y, t) \;\Rightarrow\; I_x u + I_y v = -I_t$$

Линеаризация (ф. 7.2) верна лишь на сдвигах в единицы пикселей. 16 px за кадр пирамидальный LK ещё тянет. Но смаз в 3–4 px размывает градиенты I_x, I_y, и угловые точки теряются. В лесу на 20 м те же 1,5 м/с дают в 20 раз меньший сдвиг — это и есть довод преподавателя. В статье A (Lin et al., Drones 2024) VIO теряла одометрию именно при нехватке точек и резких ускорениях.

2.3. Одинаковые проходы

Замыкание петель (гл. 10) узнаёт место по внешнему виду. Проходы склада выглядят одинаково, поэтому возможно ложное замыкание и «прыжок» позы в соседний проход. У ArUco-метки уникальный ID: одна распознанная метка сразу говорит, в каком проходе дрон.

2.4. Что уже показали наши эксперименты

В `research/landmark_ekf` одна VIO без коррекций даёт ATE 0,20 м (макс. 0,39 м) при обычном дрейфе и 0,47 м (макс. 1,02 м) при плохом. Допуск для привязки к ячейке — ±0,45 м, целевая точность — 10 см. Значит, без абсолютных меток одометрия задачу не решает.

3. Чем навигируются indoor-дроны и какой компромисс выбрать

Компромисс — два слоя. Абсолютный слой — ArUco-метки: каждая даёт полную позу без накопления ошибки. Относительный слой — IMU + оптический поток вниз + дальномер: он держит позу 1–3 с между метками. VIO — необязательное усиление второго слоя, а не основа.

Способ	Что измеряет	Сильная сторона	Слабость на складе	Роль в ВКР
ArUco / AprilTag	Абсолютная поза 6 DoF по одной метке	Нет дрейфа, уникальный ID, печать на бумаге	Нужна прямая видимость и план размещения	**Основа**
Оптический поток вниз + ToF-дальномер	Скорость над полом, высота	Дёшево, в PX4 «из коробки», пол далеко — поток малый	Гладкий бетонный пол беден текстурой	**Мост между метками**
IMU	Ускорения, угловые скорости	200–1000 Гц, всегда есть	Ошибка позиции растёт как t^2	Прогноз в фильтре

Способ	Что измеряет	Сильная сторона	Слабость на складе	Роль в ВКР
VIO (OpenVINS, VINS, QVIO)	Относительная поза 6 DoF	Работает без инфраструктуры	Смаз и быстрый поток у стеллажей (разд. 2)	Опция, не основа
Visual SLAM (ORB-SLAM3, RTAB-Map)	Поза + карта	Не нужен план	Карта уже известна; ложные замыкания	Не берём
Лидар + карта (AMCL)	Поза по геометрии	Не зависит от света	Вдоль прохода геометрия однородна; масса, цена	Альтернатива для обзора
UWB-якоря	Абсолютные XY	Нет дрейфа по XY	Металл стеллажей — многолучёвость; якорям нужно питание	Не берём
Motion capture (Vicon, OptiTrack)	Поза с точностью до миллиметра	Эталон точности	Только лаборатория	Ground truth на стенде, если доступен

Источники таблицы: статья A (Lin et al., Drones 2024) сравнивала UWB, QVIO, AprilTag и RTAB-Map и признала AprilTag самой стабильной глобальной коррекцией. Учебный квадрокоптер COEX «Клевер» на PX4 летает в помещении именно по карте ArUco-маркеров — то есть подход проверен на практике.

Почему нужен второй слой. Без него дрон «слепнет», как только метка уходит из кадра. Один IMU дрейфует быстро: смещение нуля акселерометра b_a дважды интегрируется в ошибку положения.

```
\delta p(t) = \tfrac{1}{2}\, b_a\, t^2
```

При b_a = 0,05 м/с² (порядок для недорогих MEMS, уточнить по даташиту): 1 с — 2,5 см, 3 с — 22,5 см, 5 с — 62,5 см. Поэтому нужен оптический поток вниз: он измеряет скорость, и ошибка растёт линейно с путём, а не квадратично со временем.

```
v = h\,\big(\dot\theta_{\text{поток}} - \omega_{\text{гиро}}\big)
```

h — высота по дальномеру, $\dot\theta$ — угловая скорость сдвига картинки пола, ω — поворот самого дрона по гироскопу. Довод преподавателя работает и здесь: на высоте 3–8 м пол далеко, и сдвиг за кадр мал. Коэффициент дрейфа на метр пути k нужно измерить на стенде: от него зависит шаг меток (разд. 6).

4. ArUco-маркер: сколько информации и с какого расстояния

Для склада нужен самый крупный модуль, который ещё даёт достаточно ID. Ориентир — словари 4×4 или 5×5 при стороне метки 15–20 см. Крупный модуль читается дальше и лучше переносит смаз на 1,5 м/с.

4.1. Устройство

Маркер — квадрат со стороной S. Внутри — сетка n×n бит (чёрный модуль = 0, белый = 1), вокруг — чёрная рамка шириной в один модуль, снаружи — белое поле. Рамка нужна детектору: он ищет четырёхугольник, снимает перспективу и читает биты. Размер модуля и ёмкость:

```
s_{\text{мод}} = \frac{S}{n+2}, \qquad N_{\text{бит}} = n^2, \qquad N_{ID}
\ll 2^{n^2}
```

ID в словаре меньше, чем 2^(n²): коды подобраны так, чтобы отличаться минимум на τ бит (расстояние Хэмминга) с учётом четырёх поворотов. Тогда декодер исправляет до ⌊(τ−1)/2⌋ ошибочных бит и не путает метки между собой. Пример: AprilTag 36h11 — 36 бит, τ = 11, всего 587 кодов из 2^{36}.

Именно это имел в виду преподаватель: при фиксированном S рост n даёт больше бит, но мельче модуль.

4.2. Дальность чтения

На расстоянии d модуль занимает f·s/d пикселей (та же логика, что «Утверждение 1» черновика гл. 2 для штрихкодов). Декодеру нужно не меньше p_{min} пикселей на модуль (надёжно — около 4 px, проверить на стенде):

```
\frac{f\, s_{\text{мод}}}{d} \ge p_{\min} \;\Rightarrow\; d_{\max} =
\frac{f\, S}{(n+2)\, p_{\min}}
```

Расчёт для S = 20 см, f = 1000 px (камера 1280 px с HFOV ≈ 65°), p_{min} = 4 px:

Словарь OpenCV	Бит	Модуль, мм	d_{max}, м
DICT_4X4_50 / _250	16	33	8,3
DICT_5X5_250	25	29	7,1
DICT_6X6_250	36	25	6,3
DICT_7X7_250	49	22	5,6

4.3. Смаз — второй довод за крупный модуль

Смаз в пикселях f·v·τ/d, модуль в пикселях f·s/d. Их отношение не зависит ни от камеры, ни от расстояния:

```
\beta = \frac{v\,\tau}{s_{\text{мод}}} \le \beta_{\max}
```

При v = 1,5 м/с и τ = 8 мс смаз 12 мм: это 0,36 модуля для 4×4 и 0,54 для 7×7 (S = 20 см). В нашем виртуальном облёте (SIM-09) штрихкоды читались до β ≈ 0,8. Для ArUco порог нужно измерить так же; короткая выдержка (1–2 мс) и подсветка снимают проблему.

4.4. Сколько ID нужно

Склад из 20 проходов по 10 меток с каждой стороны — 400 меток. Два пути:

- **Все ID уникальны** — нужен словарь на ⩾ 400 кодов (5×5_1000 или 6×6_1000), модуль мельче.
- **ID повторяются по проходам** — дрон и так знает позу с точностью ~1 м, и метка распознаётся по паре «ID + ожидаемое место». Хватает 4×4_50, модуль крупнее. Уникальными остаются метки на входах в проходы, чтобы дрон мог заново найти себя после потери.

5. Как по метке получить позу дрона

Одна метка даёт полную позу камеры через PnP по четырём углам. Положение она даёт точно (сантиметры на 2–4 м), а ориентацию — грубо (до 2–3°). Поэтому в фильтр передаём пиксели углов, а не готовую позу.

5.1. От пикселей к позе камеры

Углы метки в её собственной системе известны: $P_i = (\pm S/2, \pm S/2, 0)$. Детектор даёт их пиксели u_i. Проекция — пинхол-модель (ф. 4.6, 4.8):

```
s_i\, \mathbf{u}_i = K\,(R\,P_i + t), \qquad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}
```

Поза T = (R, t) — минимум ошибки репроекции (ф. 6.36):

```
T^{*} = \arg\min_{T}\; \frac{1}{2} \sum_{i=1}^{4} \Big\| \mathbf{u}_i - \frac{1}{s_i} K\, T\, P_i \Big\|_2^2
```

Начальное приближение для плоского квадрата даёт аналитический метод IPPE (в OpenCV `SOLVEPNP_IPPE_SQUARE`), дальше — итерации Гаусса–Ньютона с якобианом из ф. 6.46.

5.2. От позы камеры к позе дрона на складе

PnP даёт T_cm — метку в системе камеры. Из карты меток известна T_wm — метка в системе склада. Из калибровки известна T_bc — камера на корпусе. Поза дрона — цепочка преобразований (гл. 2–3 книги):

```
T_{wb} = T_{wm}\; T_{cm}^{-1}\; T_{bc}^{-1}
```

Это и есть «связь с реальным миром»: таблица «ID → поза метки на складе», измеренная рулеткой или лазерным дальномером при монтаже.

5.3. Точность

Дальность следует из видимого размера d ≈ f·S/w (w — сторона метки в пикселях). При шуме углов σ_{px} ошибки по глубине и поперёк взгляда (оценка порядка):

```
\sigma_d \approx \frac{d^2\, \sigma_{px}}{f\, S}, \qquad \sigma_{\perp} \approx \frac{d\, \sigma_{px}}{f}, \qquad \delta_{\text{курс}} \approx d \cdot \delta\theta
```

Ошибка по глубине растёт как d^2. При f = 1000 px, S = 20 см, σ_{px} = 0,5 px:

d, м	σ по глубине, см	σ поперёк, см	Ошибка от 2,4° курса, см
2	1,0	0,1	8
4	4,0	0,2	17
6	9,0	0,3	25

Из таблицы два вывода:

- **Рабочая дистанция — до 3–4 м.** Метки дальше читаются (разд. 4.2), но дают слабую коррекцию.
- **Курс по одной метке ненадёжен.** В нашем эксперименте LEARN-07 PnP по этикетке A6 дала ошибку дальности < 0,35%, но ориентации — до 2,4°. Две метки в кадре с базой B дают курс с ошибкой порядка σ_{pos}/B; статья A тоже отмечает, что две метки точнее одной.

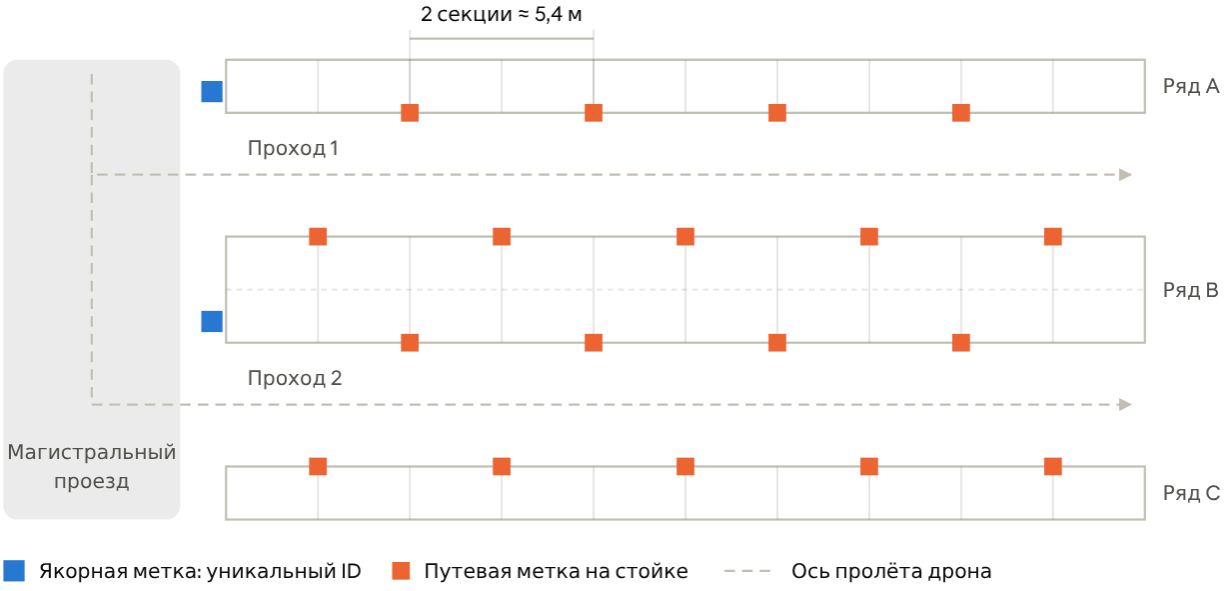
Поэтому в фильтр идут пиксели углов каждой метки, а не поза T_wb (разд. 7). Так фильтр сам взвешивает точное положение и неточный угол.

6. Где размещать метки

На каждом паллетоместе метки не нужны, а в каждом проходе — обязательны. Рекомендация: уникальная «якорная» метка на входе в каждый проход плюс «путевые» метки на стойках стеллажей через 1–2 секции (2,7–5,4 м), в шахматном порядке на обеих сторонах. Шаг следует из бюджета ошибки между метками.

Метки — на входе в проход и на стойках через 1–2 секции

Вид сверху на фрагмент склада: 3 ряда стеллажей, 2 прохода, секция ≈ 2,7 м



вид сверху · якорные и путевые метки

Дрон узнаёт проход по якорной метке на торце ряда, а внутри прохода через каждую секцию видит путевую метку — то слева, то справа.

6.1. Шаг меток из бюджета ошибки

После метки ошибка равна σ_fix, дальше растёт с путём s с коэффициентом дрейфа k (доля пути, разд. 3). До следующей метки она не должна превысить допуск ε:

$$\sigma(s) \approx \sigma_{\text{fix}} + k\, s \le \varepsilon \;\Rightarrow\; L_{\max} = \frac{\varepsilon - \sigma_{\text{fix}}}{k}$$

Допуск зависит от этапа полёта. Расчёт при σ_fix = 2 см:

Этап	Допуск ε	L_{max} при k = 1%	L_{max} при k = 3%
Перелёт по проходу (не задеть стеллаж)	30 см	28 м	9,3 м
Привязка к ячейке (±45 см — половина паллетоместа)	20 см с запасом	18 м	6 м
Подлёт к коробке (KPI проекта)	10 см	8 м	2,7 м

k для нашего датчика потока пока не измерен. Пока не измерили — берём худший случай, шаг 2,7 м (одна секция). Это согласуется с нашим экспериментом LOC-08: метки только на торцах прохода 27 м дали максимум 30–74 см, а плотные ориентиры — 6–8 см.

6.2. Видимость с борта

Боковая камера на расстоянии d от фасада видит полосу шириной W, и метка находится в кадре N кадров подряд:

$$W = 2\, d\, \tan\frac{\alpha}{2}, \qquad N = \frac{W\, F}{v}$$

При d = 1,2 м, α = 65°, F = 30 Гц, v = 1,5 м/с: W = 1,5 м, N = 30 кадров. Этого хватает с запасом, даже если часть кадров смазана. Если шаг меток L ⩽ W, в кадре всегда есть метка и дрейфа нет совсем. Такую плотность дают адресные этикетки на балках (разд. 8).

6.3. Варианты места

Место	Плюсы	Минусы	Решение
Торец ряда, вход в проход	Видна из магистрального проезда, определяет проход	Одной на проход мало (LOC-08)	Якорная метка, уникальный ID

Место	Плюсы	Минусы	Решение
Стойки (рамы) стеллажа внутри прохода	На высоте пролёта, не заслоняются паллетами	Надо монтировать на высоте	Путевые метки, основной вариант
Пол прохода (камера вниз, как у «Клевера»)	Простой монтаж	Стираются погрузчиками; на высоте 8 м нужны крупные	Только лабораторный стенд
Потолок над осью прохода (камера вверх, как в статье А)	Не заслоняются, не изнашиваются	Потолок 10–12 м, светильники слепят камеру	Альтернатива для сравнения в ВКР

Высота путевых меток — на уровне пролётов z_p, которые уже считает планировщик (`coverage`). Для прохода 27 м с шагом 5,4 м это 6 меток на сторону за один уровень высоты — листы А4, практически бесплатно.

7. Как метки соединяются с IMU и оптическим потоком

Источники соединяет расширенный фильтр Калмана (EKF, книга разд. 8.1.3). IMU и поток делают прогноз на каждом шаге, метка делает коррекцию, когда попадает в кадр. Это тот же фильтр, что уже проверен в `research/landmark_ekf`, только вместо этикеток — ArUco, а вместо VIO — поток.

7.1. Состояние и прогноз

Состояние — положение и курс на складе: $x = (p_x, p_y, \psi)$. Высоту даёт дальномер, крен и тангаж — автопилот. Вход u_k — скорость в корпусе от потока (v_b) и угловая скорость от гироскопа (ω_z):

```
p_k = p_{k-1} + R_z(\psi_{k-1})\, v_b\, \Delta t, \qquad \psi_k = \psi_{k-1} + \omega_z\, \Delta t
```

Прогноз среднего и ковариации (ф. 8.32; в обозначениях книги R — шум движения, Q — шум наблюдения):

```
\check{x}_k = f(\hat{x}_{k-1}, u_k), \qquad \check{P}_k = F\, \hat{P}_{k-1} F^{T} + R_k
```

Между метками P только растёт. Это и есть формальная запись роста σ(s) из разд. 6.

7.2. Наблюдение метки

Угол i метки j имеет известное положение на складе $P^w = T_{wm} \cdot P_i$. Ожидаемый пиксель — проекция этой точки в камеру при текущей оценке позы:

```
z_{j,i} = h(x) + v = \pi\big(K,\; T_{bc}^{-1}\, T_{wb}(x)^{-1}\,
P^{w}_{j,i}\big) + v, \qquad v \sim \mathcal{N}(0, \sigma_{px}^2 I)
```

Якобиан $H = \partial h/\partial x$ считается по цепочке: $\partial\pi/\partial P'$ из ф. 6.43 умножается на $\partial P'/\partial x$.

7.3. Коррекция и отсев ошибок

Усиление Калмана и апостериорная оценка (ф. 8.34–8.35):

```
K_k = \check{P}_k H^{T} \big(H \check{P}_k H^{T} + Q_k\big)^{-1}, \qquad
\hat{x}_k = \check{x}_k + K_k\big(z_k - h(\check{x}_k)\big), \qquad \hat{P}_k
= (I - K_k H)\, \check{P}_k
```

У EKF нет своего механизма отсева выбросов (книга, 8.1.4, п. 4). Добавляем проверку по расстоянию Махаланобиса. Для четырёх углов — 8 степеней свободы:

```
\nu = z_k - h(\check{x}_k), \qquad S = H \check{P}_k H^{T} + Q_k, \qquad
\nu^{T} S^{-1} \nu \le \chi^2_{8;\,0.99} = 20.09
```

Отсев ловит неверно распознанный ID, перекошенную или перевешенную метку и блики.

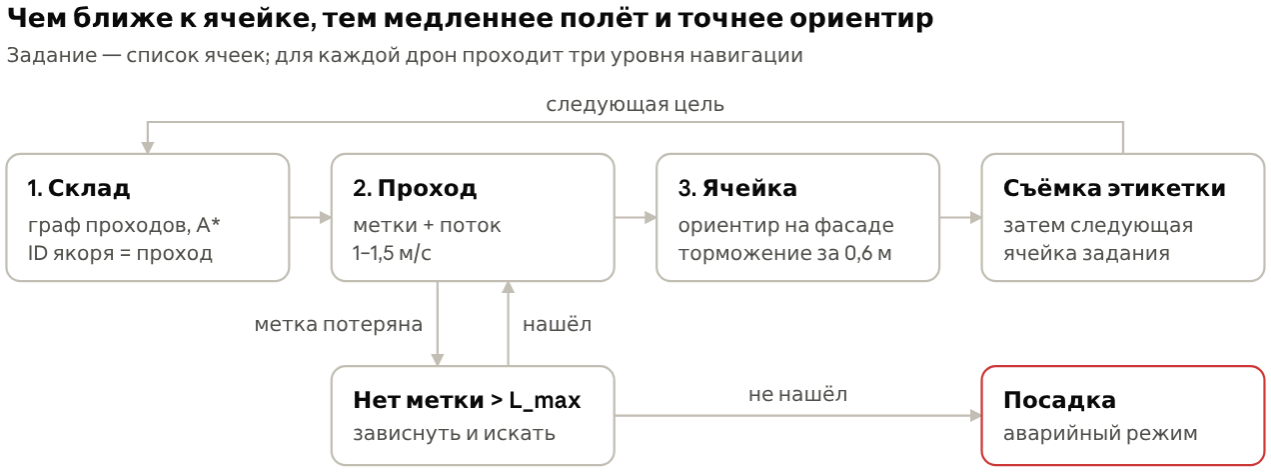
7.4. Что уже известно из наших экспериментов

- **Проверка χ^2 обязательна.** Без неё при 10% ошибочных привязок ошибка доходила до 87 см, с ней — 2,4 см (LOC-11).
- **EKF достаточно.** Оптимизация в скользящем окне (гл. 9) дала ту же точность при в 60–70 раз больших вычислениях: карта известна, модель почти линейна.
- **Ловушка редких меток.** При редких метках и сильном дрейфе фильтр становится самоуверенным и отбрасывает до 27% верных наблюдений. Лечится раздуванием P после серии отказов: доля ложных отказов падает до 9% (LOC-12). Для ArUco-меток эту опцию нужно включить.

Распределение работы: быстрый EKF2 в PX4 соединяет IMU, поток и дальномер. Узел локализации по меткам на бортовом компьютере передаёт в EKF2 исправленную позу как «внешнее зрение».

8. Как это работает: упрощённая логика

Навигационное задание выполняется на трёх уровнях точности: склад (метры), проход (десятки сантиметров), ячейка (сантиметры). На каждом уровне свой источник положения и своя скорость.



логика задания · 3 уровня, ветка потери метки

Потеря метки не останавливает задание сразу: дрон зависает и ищет её. Садится он, только если метку так и не нашёл.

8.1. Уровень 1 — навигация по складу

План склада превращается в граф: узлы — входы в проходы и перекрёстки, рёбра — оси проходов и магистрального проезда с их длинами. Маршрут — кратчайший путь по графу (Дейкстра или A*, как ROS Navigation в статье A). Дрон летит по осям на безопасной высоте, позицию даёт EKF из раздела 7. Вход в проход подтверждает якорная метка: её ID однозначно называет проход.

8.2. Уровень 2 — пролёт вдоль стеллажа

Дрон идёт по оси прохода на высоте пролёта z_p со скоростью 1–1,5 м/с. Путевые метки поправляют позу каждые 2,7–5,4 м. Если метки нет дольше расчётного шага или неопределённость P выросла выше порога — дрон зависает и ищет метку. Не нашёл — садится (это уже есть в матрице аварийных режимов проекта).

Тормозить нужно заранее, путь торможения:

```
s_{\text{торм}} = \frac{v^2}{2\, a_{\max}}
```

При v = 1,5 м/с и a_max = 2 м/с² это 0,56 м. Значит, замедление начинается не позже чем за 0,6 м до цели плюс запас на ошибку позы.

8.3. Уровень 3 — подлёт к коробке

Здесь глобальной позы мало: нужна позиция относительно конкретной ячейки. Опора — ближайший ориентир на фасаде: адресная этикетка на балке или небольшая ArUco на стойке секции. Управление строится по ошибке относительно этого ориентира (позиционное визуальное сервоуправление):

```
e = p^{*} - \hat{p}_{\text{отн}}, \qquad v_{\text{кмд}} =
\operatorname{sat}\big(K_p\, e,\; v_{\max}\big)
```

p* — нужная точка перед ячейкой, например 1,2 м от фасада напротив центра паллетоместа. p̂_отн — поза относительно ориентира по PnP. С 1–1,5 м ошибка PnP — около сантиметра (раздел 5.3). Дальше — съёмка этикетки и переход к следующей цели.

8.4. Связь с реальным миром

Что берётся с реального склада:

1. **План склада:** ряды, секции, ярусы, ширина проходов. Замер или выгрузка из WMS; модель склада уже есть в `core`.
2. **Карта меток:** таблица «ID → поза T_wm». Замер рулеткой или лазерным дальномером при монтаже.
3. **Калибровка камер:** матрица K, дисторсия, положение камеры на корпусе T_bc. Инструмент калибровки по шахматке уже есть в `tools`.
4. **Задание:** список ячеек или точек — от оператора или из WMS.

Замкнутый цикл: камера → детектор меток → EKF → поза → планировщик → уставка скорости → PX4 → движение → новый кадр. IMU работает на сотнях герц, метки — 10–30 Гц, планировщик — единицы герц.

9. Что из INVENT_DRONE остаётся, а что меняется

Переделывать с нуля не нужно: вся математика новой темы (камера, PnP, EKF с χ², модель склада) уже есть в `core` и проверена тестами. Меняется источник наблюдений (ArUco вместо этикеток) и одометрия (поток вместо VIO). Инвентаризация уходит в прикладной сценарий.

Часть проекта	Судьба	Что сделать
`localization.py` (EKF, χ², раздувание P)	Ядро новой темы	Наблюдение — 4 угла ArUco вместо центра этикетки; включить `inflate_after`

Часть проекта	Судьба	Что сделать
`camera.py`, `frames.py`, `pnp.py`	Остаются как есть	PnP переиспользуется для квадратной метки
`layout.py` (модель склада)	Остаётся	Добавить карту меток «ID → поза» и граф проходов
`research/landmark_ekf`	Главный эксперимент	Сценарии: шаг меток L × скорость v × дрейф k, поток вместо VIO
`coverage.py`, `research/sortie_planning`	Второстепенно	План облёта = навигационное задание (последовательность точек)
`barcode.py`, `association.py`, `reconcile.py`, `economics.py`	Прикладной сценарий	В ВКР — демонстрация или приложение
ROS 2: `landmark_localizer_node`, `mission_node`, `safety_node`	Остаются	Детектор ArUco (OpenCV `cv2.aruco` уже стоит в `.venv`); три уровня из разд. 8 в миссии
ROS 2: `fake_vio_node`	Заменить	Модель потока + дальномера или EKF2 PX4 с датчиком потока в SITL
`sim/generate_world.py`	Доработать	Метки на стойках и торцах в мире Gazebo
VIO: OpenVINS / Basalt (ADR-5, LOC-03)	Убрать с критического пути	Опция для сравнения в главе «Обзор»
ADR-4 (локализация)	Пересмотреть	«Основа — ArUco + поток; адресные этикетки — точная привязка у ячейки»

Кандидаты в научную новизну новой темы:

1. Методика выбора параметров меток (размер, словарь, шаг, высота) по требуемой точности и скорости полёта — формулы разделов 4–6 с проверкой в симуляции.
2. Двухслойная локализация «метки + поток» с адаптивным порогом при редких метках.

3. Экспериментальная зависимость точности навигации от шага меток и скорости — ответ на вопрос «на каждом ли стеллаже» цифрами.

10. Открытые вопросы и план

Вопросы к преподавателю:

- Точная формулировка темы.
- Инвентаризация — отдельная глава или только демонстрация в экспериментах?
- Допустима ли VIO как необязательное усиление, или работа должна обойтись без неё?
- Достаточно ли симуляции и лабораторного стенда, или нужен реальный склад?

Технические вопросы (решаются замером):

- Дрейф потока k на нашем датчике и полу. Он задаёт шаг меток.
- p_{min} и допустимый смаз β_{max} для ArUco. Они задают словарь и размер.
- Какая камера смотрит на метки: боковые (как для этикеток) или отдельная фронтальная.

Шаги:

- [ ] Получить точное название темы и согласовать схему «метки + поток, без SLAM в основе»
- [ ] Распечатать ArUco 4×4 и 5×5 (S = 15 и 20 см), замерить дальность, смаз и точность PnP на камере стенда
- [ ] Добавить метки в мир Gazebo и поток + дальномер в SITL вместо fake VIO
- [ ] Главный эксперимент: ошибка навигации при шаге меток 2,7 / 5,4 / 8,1 / 27 м и скорости 0,5 / 1 / 1,5 м/с
- [ ] После утверждения темы — переписать план ВКР, мини-обзор и ADR-4

Источники: Gao X., Zhang T. *Introduction to Visual SLAM: From Theory to Practice*, Springer, 2021 (ф. 1.1–1.2, 4.5–4.12, 6.36, 6.43–6.46, 7.1–7.9, 8.32–8.35); Lin H.-Y. et al. *Development of UAV Navigation and Warehouse Inventory System Based on Reinforcement Learning*, Drones 2024, 8(6), 220; журнал экспериментов проекта (`docs/notes/experiments.md`, `research/landmark_ekf`). Характеристики словарей ArUco, камер и MEMS в расчётах — типовые значения по памяти, их нужно сверить с даташитами выбранного железа.